LAPORAN PRATIKUM PEMROGRAMAN
PRAKTIKUM 6
MENGEMBANGKAN FUNGSI-FUNGSI SENDIRI



Disusun Oleh :

NAMA	:	Muhammad Aditya Rahmansyah
NRP	:	3226600122
KELAS	:	ST.r Tekkom A
MATA KULIAH	:	Pemrograman
DOSEN	:	Ir. Sigit Wasista, M.Kom.
ASSISTEN DOSEN	:	Rofiulloh Mu'azin Al-insof, A.Md.
TANGGAL PRATIKUM	:	Kamis, 28 Oktober 2026

PROGRAM STUDI SARJANA TERAPAN TEKNIK KOMPUTER
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

BAB I. PENDAHULUAN

1.1 Latar Belakang

Fungsi adalah bagian program bernama yang dirancang untuk menjalankan tugas tertentu. Dalam pemrograman C++, fungsi memungkinkan program besar dipecah menjadi bagian-bagian kecil yang lebih mudah dibuat, diuji, dan dipelihara. Dengan fungsi, kode yang sama tidak perlu ditulis berulang, cukup dipanggil dari bagian program mana pun.

1.2 Tujuan Praktikum

• Memahami fungsi sebagai bagian program yang menjalankan tugas tertentu.

• Membedakan prototipe, definisi, pemanggilan fungsi, parameter, dan argumen.

• Membuat fungsi void dan fungsi yang mengembalikan nilai.

• Menggunakan fungsi dengan parameter serta nilai balik.

• Memahami variabel lokal, global, otomatis, dan lokal static.

• Menguji program dengan kasus normal, batas, dan masukan tidak valid.

1.3 Ruang Lingkup

Materi yang dipraktikkan meliputi prototipe fungsi, definisi fungsi, pemanggilan fungsi, parameter dan argumen, fungsi void dan fungsi dengan nilai balik, fungsi yang memanggil fungsi lain, rekursi, serta variabel lokal, otomatis, dan lokal static. Praktikum terdiri dari enam percobaan dan sepuluh tugas yang masing-masing diuji dengan kasus normal, batas, dan masukan tidak valid.

BAB II. DASAR TEORI

2.1 Pengertian Fungsi

Fungsi merupakan bagian program bernama yang dirancang untuk menjalankan tugas tertentu. Fungsi memecah masalah menjadi bagian yang lebih kecil, mengurangi pengulangan kode, dan memudahkan pengujian serta pemeliharaan program.

2.2 Prototipe, Definisi, dan Pemanggilan Fungsi

Prototipe menyatakan antarmuka fungsi agar compiler mengetahui tipe nilai balik dan parameternya. Definisi berisi perintah yang dijalankan. Pemanggilan fungsi menjalankan fungsi tersebut.

2.3 Parameter, Argumen, dan Nilai Balik

Parameter adalah variabel dalam daftar parameter fungsi, sedangkan argumen adalah nilai yang dikirim saat fungsi dipanggil.

2.4 Fungsi `void` dan Fungsi dengan Nilai Balik

Fungsi bertipe void tidak mengembalikan nilai. Fungsi bertipe int, double, bool, atau tipe lain dapat mengembalikan nilai menggunakan return.

2.5 Variabel Lokal, Global, dan `static`

Variabel lokal merupakan variabel yang hanya bisa diakses diruang lingkup tempat variabel dideklarasikan. Variabel global merupakan variabel yang dideklarasikan diluar fungsi. Variabel static merupakan variabel yang memiliki durasi penyimpanan sepanjang program berjalan.

BAB III. PELAKSANAAN PRAKTIKUM

3.1 Perangkat dan Lingkungan

- Perangkat komputer: MSI Modern 14 B5m
- Sistem operasi: Windows 11
- Compiler/IDE: GCC (MinGW), Dev-C++

3.2 Langkah Kerja

1. Menulis kode program setiap percobaan dan tugas dalam berkas .cpp memakai editor Visual Studio Code.

2. Mengompilasi program dengan compiler GCC (MinGW) dan memperbaiki kesalahan kompilasi jika ada.

3. Menjalankan program dan memasukkan data uji sesuai kasus normal, batas, dan tidak valid.

4. Menangkap layar kode program dan hasil eksekusi sebagai bukti.

5. Mencatat hasil pada tabel pengujian, lalu menganalisis cara kerja fungsi dan kesesuaian keluaran.

3.3 Ringkasan Percobaan

• Percobaan 1 – Fungsi tanpa parameter dan tanpa nilai balik (tampilkanPesan).

• Percobaan 2 – Fungsi dengan parameter (jumlahSampai).

• Percobaan 3 – Fungsi FPB dengan nilai balik (algoritma Euclid).

• Percobaan 4 – Fungsi memanggil fungsi lain (nilaiAbsolut dan akarKuadrat).

• Percobaan 5 – Fungsi nilai minimum (minimum).

• Percobaan 6 – Variabel otomatis dan static (demo).

• Tugas 6.5.1 sampai 6.5.10 – kabisat, faktorial, permutasi dan kombinasi, konversi suhu, bilangan prima, variabel otomatis dan static, nilai terbesar, validasi segitiga, rekursi jumlah digit, dan analisis digit.

BAB IV. HASIL DAN ANALISIS

Gunakan format berikut untuk **setiap percobaan**.

4.1 Percobaan 1 – Fungsi tanpa parameter dan tanpa nilai balik

A. Tujuan

Membuat program yang memanggil tampilkanPesan() sebanyak lima kali. Perhatikan prototipe sebelum main() dan definisi setelah main().

B. Algoritma atau Langkah Kerja

1. Sertakan header <iostream>
2. Tulis prototipe fungsi void tampilkanPesan() sebelum main().
3. Di dalam main(), ulangi untuk i = 1 sampai i = 5: cetak teks "Pesan ke-i: ", lalu panggil tampilkanPesan().
4. Definisi tampilkanPesan() (ditulis setelah main()) mencetak teks "Belajar fungsi C++" diikuti baris baru.
5. Setelah perulangan selesai, main() mengembalikan 0 dan program berakhir.

C. Kode Program

```cpp
#include <iostream>

void tampilkanPesan();

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::cout << "Pesan ke-" << i << ": ";
        tampilkanPesan();
    }
    return 0;
}

void tampilkanPesan() {
    std::cout << "Belajar fungsi C++\n";
}
```

D. Hasil Eksekusi

```
Pesan ke-1: Belajar fungsi C++
Pesan ke-2: Belajar fungsi C++
Pesan ke-3: Belajar fungsi C++
Pesan ke-4: Belajar fungsi C++
Pesan ke-5: Belajar fungsi C++
```

E. Pengujian

No	Data uji	Output aktual	Keterangan
1	Pemanggilan dalam perulangan i = 1 sampai 5 (normal)	Pesan ke-1: Belajar fungsi C++ Pesan ke-2: Belajar fungsi C++ Pesan ke-3: Belajar fungsi C++ Pesan ke-4: Belajar fungsi C++ Pesan ke-5: Belajar fungsi C++	Sesuai. Fungsi dipanggil tepat 5 kali.
2	Iterasi pertama (i = 1) dan terakhir (i = 5) (batas)	Baris pertama "Pesan ke-1", baris terakhir "Pesan ke-5", tidak ada "Pesan ke-6"	Sesuai. Batas perulangan benar.
3	Jumlah baris keluaran (lainnya)	5 baris, tiap pesan di baris terpisah	Sesuai. \n pada fungsi memisahkan tiap pesan.

F. Analisis

Fungsi tampilkanPesan() tidak memiliki parameter dan bertipe void, sehingga tidak menerima argumen dan tidak mengembalikan nilai. Di dalam main(), perulangan for menjalankan pemanggilan tampilkanPesan() sebanyak lima kali, dan sebelum tiap pemanggilan main() mencetak nomor pesan.

Prototipe di atas main() memberi tahu compiler bahwa fungsi tersebut ada, sehingga definisinya boleh diletakkan setelah main(). Keluaran berisi lima baris karena fungsi dipanggil lima kali, dengan nomor pesan yang berasal dari variabel i milik main().

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Hal yang perlu diperhatikan: jika prototipe dihapus padahal definisi berada setelah main(), compiler akan melaporkan error bahwa tampilkanPesan belum dideklarasikan. Solusinya adalah menuliskan prototipe sebelum main().

4.2 Percobaan 2 – Fungsi tanpa parameter dan tanpa nilai balik

A. Tujuan

Membuat fungsi jumlahSampai(int n) yang menghitung jumlah bilangan bulat dari 1 sampai n, menjalankannya untuk n = 10, 20, dan 50, serta menjelaskan hubungan antara argumen dan parameter.

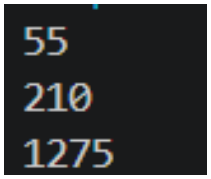
B. Algoritma atau Langkah Kerja

1. Sertakan header <iostream>
2. Definisikan fungsi int jumlahSampai(int n) dengan parameter n.
3. Di dalam fungsi: jumlah = 0; untuk i = 1 sampai n lakukan jumlah = jumlah + i.
4. Kembalikan jumlah dengan return.
5. Di main(), panggil jumlahSampai(10), jumlahSampai(20), dan jumlahSampai(50), lalu cetak hasilnya.
6. Selesai

C. Kode program

```cpp
#include <iostream>
int jumlahSampai(int n) {
    int jumlah = 0;
    for (int i = 1; i <= n; ++i) {
        jumlah += i;
    }
    return jumlah;
}
int main() {
    std::cout << jumlahSampai(10) << '\n';
    std::cout << jumlahSampai(20) << '\n';
    std::cout << jumlahSampai(50) << '\n';
    return 0;
}
```

D. Hasil Eksekusi



E. Pengujian

No	Data uji	Output aktual	Keterangan
1	jumlahSampai(10) (normal)	55	Sesuai. 10 × 11 / 2 = 55.
2	jumlahSampai(20) (normal)	210	Sesuai. 20 × 21 / 2 = 210.
3	jumlahSampai(50) (nilai lebih besar)	1275	Sesuai. 50 × 51 / 2 = 1275.

F. Analisis

Angka 10, 20, dan 50 yang ditulis pada pemanggilan adalah argumen, sedangkan n pada definisi fungsi adalah parameter. Saat fungsi dipanggil, nilai argumen disalin ke parameter n (pass by value), sehingga perubahan n di dalam fungsi tidak memengaruhi nilai di pemanggil.

Variabel jumlah adalah variabel lokal yang dibuat ulang dan diisi 0 pada setiap pemanggilan, sehingga hasil satu pemanggilan tidak terbawa ke pemanggilan berikutnya. Perulangan menjumlahkan 1 sampai n, lalu return mengirim hasilnya kembali ke main() untuk dicetak. Hasil 55, 210, dan 1275 sesuai dengan rumus n(n+1)/2.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Karena fungsi didefinisikan sebelum main(), prototipe tidak diperlukan. Perlu diingat bahwa untuk n = 0 atau n negatif perulangan tidak berjalan sehingga hasilnya 0.

4.3 Percobaan 3 – Fungsi FPB dengan nilai balik

A. Tujuan

Mengimplementasikan algoritma Euclid dalam fungsi fpb(int a, int b) yang menerima dua bilangan bulat positif dan mengembalikan Faktor Persekutuan Terbesar (FPB).

B. Algoritma atau Langkah Kerja

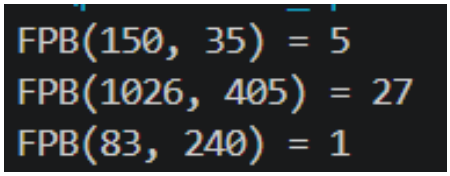
1. Sertakan header <iostream>
2. Definisikan fungsi int fpb(int a, int b).
3. Selama b ≠ 0: sisa = a mod b; a = b; b = sisa.
4. Ketika b = 0, kembalikan a sebagai FPB.
5. Di main(), panggil fpb(150, 35), fpb(1026, 405), dan fpb(83, 240), lalu cetak hasilnya.

C. Kode program

```cpp
#include <iostream>

int fpb(int a, int b) {
    while (b != 0) {
        int sisa = a % b;
        a = b;
        b = sisa;
    }
    return a;
}
int main() {
    std::cout << "FPB(150, 35) = " << fpb(150, 35) << '\n';
    std::cout << "FPB(1026, 405) = " << fpb(1026, 405) << '\n';
    std::cout << "FPB(83, 240) = " << fpb(83, 240) << '\n';
    return 0;
}
```

D. Hasil Eksekusi



E. Pengujian

No	Data uji	Output aktual	keterangan
1	fpb(150, 35) (normal)	FPB(150, 35) = 5	Sesuai. 150 mod 35 = 10, 35 mod 10 = 5, 10 mod 5 = 0.
2	fpb(1026, 405) (normal)	FPB(1026, 405) = 27	Sesuai. 1026 = 2 × 27 × 19 dan 405 = 27 × 15.
3	fpb(83, 240) (batas: saling prima)	FPB(83, 240) = 1	Sesuai. 83 prima dan tidak membagi 240.

F. Analisis

Fungsi fpb() menerima dua parameter bertipe int dan mengembalikan satu nilai int melalui return. Argumen yang dikirim dari main() disalin ke parameter a dan b, sehingga perubahan a dan b di dalam perulangan tidak mengubah nilai asli di pemanggil.

Algoritma Euclid memakai sifat FPB(a, b) = FPB(b, a mod b). Perulangan terus mengganti pasangan (a, b) dengan (b, sisa) sampai b bernilai 0, dan saat itu a adalah FPB. Untuk 83 dan 240 sisa pembagian akhirnya 1, sehingga keduanya saling prima.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Fungsi ini mengasumsikan masukan bilangan bulat positif; jika b = 0 sejak awal fungsi langsung mengembalikan a, dan jika keduanya 0 hasilnya 0.

4.4 Percobaan 4 – Fungsi memanggil fungsi lain

A. Tujuan

Melengkapi fungsi nilaiAbsolut() dan akarKuadrat() dengan metode Newton, serta menunjukkan bahwa sebuah fungsi dapat memanggil fungsi lain dan menangani masukan negatif dan nol.

B. Algoritma atau Langkah Kerja

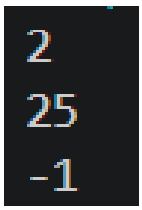
1. Mulai, sertakan header <iostream>.
2. nilaiAbsolut(x): jika x < 0 kembalikan -x, selain itu kembalikan x.
3. akarKuadrat(x): jika x < 0 kembalikan -1.0 (penanda masukan negatif); jika x = 0 kembalikan 0.0.
4. Set tebakan = x dan epsilon = 0,00001.
5. Selama nilaiAbsolut(tebakan × tebakan − x) ≥ epsilon: tebakan = (tebakan + x / tebakan) / 2.
6. Kembalikan tebakan. Di main(), panggil akarKuadrat(4.0), akarKuadrat(625.0), dan akarKuadrat(-39.5).

C. Kode program

```cpp
#include <iostream>

double nilaiAbsolut(double x) {
    return (x < 0) ? -x : x;
}
double akarKuadrat(double x) {
    if (x < 0) return -1.0;
    if (x == 0) return 0.0;
    double tebakan = x;
    const double epsilon = 0.00001;
    while (nilaiAbsolut(tebakan * tebakan - x) >= epsilon) {
        tebakan = (tebakan + x / tebakan) / 2.0;
    }
    return tebakan;
}
int main() {
    std::cout << akarKuadrat(4.0) << '\n';
    std::cout << akarKuadrat(625.0) << '\n';
    std::cout << akarKuadrat(-39.5) << '\n';
    return 0;
}
```

D. Hasil Eksekusi



E. Pengujian

No	Data Uji	Output aktual	Keterangan

1	akarKuadrat(4.0) (normal)	2	Sesuai. √4 = 2.
2	akarKuadrat(625.0) (normal, nilai besar)	25	Sesuai. √625 = 25.
3	akarKuadrat(-39.5) (tidak valid)	-1	Sesuai. -1 adalah penanda masukan negatif.

F. Analisis

Fungsi akarKuadrat() memanggil nilaiAbsolut() pada kondisi perulangan untuk mengukur selisih antara kuadrat tebakan dan x. Ini menunjukkan bahwa fungsi dapat memanggil fungsi lain agar kode lebih modular. Argumen x disalin ke parameter x, dan nilai akhir tebakan dikembalikan melalui return.

Metode Newton memperbarui tebakan dengan rumus (tebakan + x / tebakan) / 2 dan konvergen dengan cepat. Perulangan berhenti ketika selisih kuadrat tebakan terhadap x kurang dari epsilon. Hasil 2 dan 25 tampil sebagai bilangan bulat karena cout hanya menampilkan 6 digit signifikan, sedangkan -1 muncul karena masukan negatif ditolak dengan nilai penanda.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Penanda -1.0 harus diperiksa oleh pemanggil, karena jika tidak, -1 dapat salah dianggap sebagai hasil akar kuadrat. Kasus x = 0 ditangani lebih dulu agar tidak terjadi pembagian dengan nol pada x / tebakan.

4.5 Percobaan 5 – Fungsi nilai minimum

A. Tujuan

Menjalankan fungsi minimum() untuk membandingkan dua bilangan double, dengan pengujian bilangan positif, negatif, dua bilangan yang sama, dan masukan tidak valid.

().

B. Algoritma atau Langkah Kerja

1. Sertakan header <iostream>
2. Definisikan fungsi double minimum(double x, double y): jika x < y kembalikan x, selain itu kembalikan y.
3. Di main(), tampilkan permintaan masukan dan baca dua bilangan a dan b.
4. Jika pembacaan gagal, cetak "Masukan tidak valid" dan akhiri program dengan kode 1.
5. Jika berhasil, cetak "Nilai minimum = " diikuti minimum(a, b).

C. Kode program

```cpp
#include <iostream>

double minimum(double x, double y) {
    if (x < y) return x;
    return y;
}
int main() {
    double a, b;
    std::cout << "Masukkan dua bilangan: ";
    if (!(std::cin >> a >> b)) {
        std::cout << "Masukan tidak valid\n";
        return 1;
    }
    std::cout << "Nilai minimum = " << minimum(a, b) << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
Masukkan dua bilangan: 10 90
Nilai minimum = 10
```

```
Masukkan dua bilangan: [ [
Masukan tidak valid
```

E. Pengujian

No	Data uji	Output aktual	Keterangan
1	10 dan 90 (bilangan positif)	Nilai minimum = 10	Sesuai.
2	-5 dan -8 (bilangan negatif)	Nilai minimum = -8	Sesuai. -8 lebih kecil dari -5.
3	7 dan 7 (dua bilangan sama)	Nilai minimum = 7	Sesuai. Kedua nilai sama sehingga hasilnya tetap 7.
4	[[(masukan bukan angka)	Masukan tidak valid	Sesuai. Program menolak masukan dan keluar.

F. Analisis

Fungsi minimum() memiliki dua parameter double dan mengembalikan nilai yang lebih kecil. Nilai a dan b dari main() menjadi argumen yang disalin ke x dan y. Jika x < y fungsi mengembalikan x, dan pada semua kondisi lain (y lebih kecil atau keduanya sama) fungsi mengembalikan y.

Ekspresi std::cin >> a >> b bernilai false ketika masukan tidak dapat diubah menjadi bilangan, sehingga program menampilkan pesan kesalahan dan keluar dengan return 1. Karena itu masukan seperti "[[" tidak pernah sampai ke fungsi minimum().

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala yang diantisipasi adalah masukan non-angka, yang diatasi dengan memeriksa hasil std::cin sebelum memanggil fungsi.

4.6 Percobaan 6 – Variabel otomatis dan static

A. Tujuan

Memanggil fungsi demo() sebanyak tiga kali dan membedakan perilaku variabel otomatis dengan variabel lokal static pada setiap pemanggilan.

B. Algoritma atau Langkah Kerja

1. Mulai, sertakan header <iostream>.
2. Definisikan fungsi void demo() dengan variabel otomatis int otomatis = 0 dan variabel static int tetap = 0.
3. Naikkan kedua variabel dengan ++ lalu cetak nilainya.
4. Di main(), panggil demo() sebanyak tiga kali dengan perulangan for.
5. Catat nilai otomatis dan static pada tiap pemanggilan.

C. Kode program

```cpp
#include <iostream>

void demo() {
    int otomatis = 0;
    static int tetap = 0;
    ++otomatis;
    ++tetap;
    std::cout << "otomatis=" << otomatis
              << ", static=" << tetap << '\n';
}
int main() {
    for (int i = 0; i < 3; ++i) demo();
    return 0;
}
```

D. Hasil Eksekusi

```
otomatis=1, static=1
otomatis=1, static=2
otomatis=1, static=3
```

E. Pengujian

No	Data uji	Output aktual	Keterangan
1	Pemanggilan demo() ke-1	otomatis=1, static=1	Sesuai. Kedua variabel baru dibuat.
2	Pemanggilan demo() ke-2	otomatis=1, static=2	Sesuai. Variabel static mempertahankan nilai.
3	Pemanggilan demo() ke-3	otomatis=1, static=3	Sesuai. Variabel otomatis kembali dibuat dari 0.

F. Analisis

Variabel otomatis dibuat setiap fungsi dipanggil dan dihapus saat fungsi selesai, sehingga selalu dimulai dari 0 dan menjadi 1 setelah ++. Variabel static hanya diinisialisasi satu kali, pada pemanggilan pertama, dan nilainya dipertahankan di antara pemanggilan.

Akibatnya nilai otomatis selalu 1, sedangkan nilai static bertambah 1, 2, 3 sesuai jumlah pemanggilan. Walaupun hidup selama program berjalan, variabel static tetap hanya bisa diakses dari dalam fungsi demo().

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala konsep yang perlu diingat adalah inisialisasi static int tetap = 0 hanya dijalankan satu kali, sehingga bukan berarti nilainya di-reset setiap pemanggilan.

4.7 Tugas 6.5.1 – Fungsi kabisat

A. Tujuan

Membuat fungsi bool kabisat(int tahun) yang mengembalikan true jika tahun kabisat dan false jika bukan, berdasarkan aturan kalender Gregorian.

B. Algoritma atau Langkah Kerja

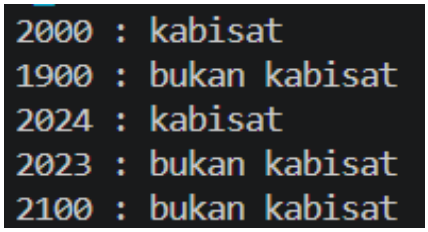
1. Suatu tahun kabisat jika habis dibagi 400, atau habis dibagi 4 tetapi tidak habis dibagi 100.
2. Fungsi kabisat(tahun) mengembalikan hasil ekspresi (tahun % 400 == 0) || (tahun % 4 == 0 && tahun % 100 != 0).
3. Di main(), simpan data uji {2000, 1900, 2024, 2023, 2100} dalam array.
4. Untuk tiap tahun, panggil kabisat() lalu cetak "kabisat" atau "bukan kabisat".

C. Kode program

```cpp
#include <iostream>

bool kabisat(int tahun) {
    return (tahun % 400 == 0) || (tahun % 4 == 0 && tahun % 100 != 0);
}
int main() {
    int uji[] = {2000, 1900, 2024, 2023, 2100};
    for (int t : uji)
        std::cout << t << (kabisat(t) ? " : kabisat" : " : bukan kabisat") << '\n';
    return 0;
}
```

D. Hasil Eksekusi



E. Pengujian

No.	Data Uji	Output aktual	Keterangan
1	2024 (normal, kabisat)	2024 : kabisat	Sesuai. 2024 habis dibagi 4 dan tidak habis dibagi 100.
2	2023 (normal, bukan kabisat)	2023 : bukan kabisat	Sesuai. 2023 tidak habis dibagi 4.
3	2000 (batas, kelipatan 400)	2000 : kabisat	Sesuai. Kelipatan 400 tetap kabisat.
4	1900 (batas, kelipatan 100 bukan 400)	1900 : bukan kabisat	Sesuai. Kelipatan 100 tetapi bukan 400.
5	2100 (batas, kelipatan 100 bukan 400)	2100 : bukan kabisat	Sesuai.

F. Analisis

Fungsi kabisat() menerima satu parameter int dan mengembalikan nilai bool lewat return. Ekspresi logikanya menerapkan aturan Gregorian: kelipatan 400 selalu kabisat, kelipatan 4 kabisat kecuali kelipatan 100.

Tahun 2000 kabisat karena habis dibagi 400, sedangkan 1900 dan 2100 bukan kabisat karena habis dibagi 100 tetapi tidak habis dibagi 400. Tahun 2024 kabisat dan 2023 bukan, sesuai aturan.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala logika yang mungkin muncul adalah hanya memakai aturan "habis dibagi 4", yang salah untuk 1900 dan 2100. Hal ini diatasi dengan menambahkan syarat kelipatan 100 dan 400 serta tanda kurung agar urutan || dan && benar.

4.8 Tugas 6.5.2 – Faktorial

A. Tujuan

Membuat fungsi faktorial(int n) untuk menghitung n! bagi n ≥ 0, memvalidasi masukan negatif, dan menjelaskan batas tipe data yang digunakan.

B. Algoritma atau Langkah Kerja

1. Fungsi faktorial(n, valid) mengisi valid = true, lalu jika n < 0 atau n > 20, set valid = false dan kembalikan 0.
2. Set hasil = 1 (bertipe unsigned long long).
3. Untuk i = 2 sampai n, lakukan hasil = hasil × i.
4. Kembalikan hasil.
5. Di main(), uji n = 0, 1, 5, 10, 20, 21, dan -3; cetak hasil jika valid, atau pesan kesalahan jika tidak.

C. Kode program

```cpp
#include <iostream>

unsigned long long faktorial(int n, bool &valid) {
    valid = true;
    if (n < 0 || n > 20) { valid = false; return 0; }
    unsigned long long hasil = 1;
    for (int i = 2; i <= n; ++i) hasil *= i;
    return hasil;
}
int main() {
    int uji[] = {0, 1, 5, 10, 20, 21, -3};
    for (int n : uji) {
        bool ok;
        unsigned long long f = faktorial(n, ok);
        if (ok) std::cout << n << "! = " << f << '\n';
        else    std::cout << n << "! : masukan tidak valid / melebihi batas (maks 20)\n";
    }
    return 0;
}
```

D. Hasil Eksekusi

```
2_faktorial.exe"
0! = 1
1! = 1
5! = 120
10! = 3628800
20! = 2432902008176640000
21! : masukan tidak valid / melebihi batas (maks 20)
-3! : masukan tidak valid / melebihi batas (maks 20)
```

E. Pengujian

No.	Data Uji	Output aktual	Keterangan
1	n = 0 (batas bawah)	0! = 1	Sesuai. Menurut definisi 0! = 1.

2	n = 5 (normal)	5! = 120	Sesuai. 5 × 4 × 3 × 2 × 1 = 120.
3	n = 10 (normal)	10! = 3628800	Sesuai.
4	n = 20 (batas atas)	20! = 2432902008176640000	Sesuai. Masih muat di unsigned long long.
5	n = 21 (melebihi batas)	21! : masukan tidak valid / melebihi batas (maks 20)	Sesuai. Ditolak untuk mencegah overflow.
6	n = -3 (tidak valid)	-3! : masukan tidak valid / melebihi batas (maks 20)	Sesuai. Faktorial negatif tidak didefinisikan.

F. Analisis

Fungsi faktorial() menerima n sebagai parameter dan parameter kedua berupa referensi bool &valid untuk memberi tahu pemanggil apakah masukan sah. Dengan referensi, perubahan valid di dalam fungsi langsung terlihat di main(), sedangkan nilai faktorial dikembalikan lewat return.

Tipe unsigned long long (64 bit) memiliki nilai maksimum 18.446.744.073.709.551.615 (sekitar 1,8 × 10^19). Nilai 20! = 2.432.902.008.176.640.000 masih muat, tetapi 21! = 51.090.942.171.709.440.000 melampaui batas itu dan akan mengalami overflow. Karena itu n > 20 ditolak, sama seperti n negatif.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala utamanya adalah overflow pada n > 20, yang diatasi dengan memakai unsigned long long dan membatasi n maksimum 20. Pemberitahuan masukan tidak sah dilakukan lewat parameter referensi valid, karena nilai kembalian 0 tidak bisa dibedakan dari hasil yang sah.

4.9 Tugas 6.5.3 – Permutasi dan kombinasi

A. Tujuan

Membuat fungsi permutasi(n, r) dan kombinasi(n, r) berdasarkan rumus P(n, r) = n!/(n−r)! dan C(n, r) = n!/[r!(n−r)!], dengan validasi 0 ≤ r ≤ n.

B. Algoritma atau Langkah Kerja

1. validNR(n, r) bernilai true jika n ≥ 0, r ≥ 0, dan r ≤ n.
2. permutasi(n, r): jika tidak valid kembalikan -1; hasil = 1; untuk i = 0 sampai r − 1, hasil = hasil × (n − i); kembalikan hasil.
3. kombinasi(n, r): jika tidak valid kembalikan -1; jika r > n − r, set r = n − r; hasil = 1; untuk i = 1 sampai r, hasil = hasil × (n − r + i) / i; kembalikan hasil.
4. Di main(), uji pasangan (5,2), (10,3), (5,0), (5,5), (3,5), dan (-1,0); cetak P dan C, atau pesan tidak valid jika nilainya negatif.

C. Kode program

```cpp
#include <iostream>

bool validNR(int n, int r) { return n >= 0 && r >= 0 && r <= n; }


long long permutasi(int n, int r) {
    if (!validNR(n, r)) return -1;
    long long hasil = 1;
    for (int i = 0; i < r; ++i) hasil *= (n - i);
    return hasil;
}

long long kombinasi(int n, int r) {
    if (!validNR(n, r)) return -1;
    if (r > n - r) r = n - r;
    long long hasil = 1;
    for (int i = 1; i <= r; ++i) hasil = hasil * (n - r + i) / i;
    return hasil;
}

int main() {
    int uji[][2] = {{5,2},{10,3},{5,0},{5,5},{3,5},{-1,0}};
    for (auto &u : uji) {
        long long p = permutasi(u[0], u[1]), c = kombinasi(u[0], u[1]);
        std::cout << "n=" << u[0] << ", r=" << u[1] << " : ";
        if (p < 0) std::cout << "tidak valid (harus 0 <= r <= n)\n";
        else std::cout << "P=" << p << ", C=" << c << '\n';
    }
    return 0;
}
```

D. Hasil Eksekusi

```
n=5, r=2 : P=20, C=10
n=10, r=3 : P=720, C=120
n=5, r=0 : P=1, C=1
n=5, r=5 : P=120, C=1
n=3, r=5 : tidak valid (harus 0 <= r <= n)
n=-1, r=0 : tidak valid (harus 0 <= r <= n)
```

E. Pengujian

No	Data Uji	Output aktual	Keterangan
1	n = 5, r = 2 (normal)	P=20, C=10	Sesuai. P = 5 × 4 = 20 dan C = 20 / 2 = 10.

2	n = 10, r = 3 (normal)	P=720, C=120	Sesuai. P = 10 × 9 × 8 = 720 dan C = 720 / 6 = 120.
3	n = 5, r = 0 (batas bawah)	P=1, C=1	Sesuai. Tidak ada elemen yang dipilih.
4	n = 5, r = 5 (batas atas)	P=120, C=1	Sesuai. P = 5! = 120 dan hanya ada 1 cara memilih semua.
5	n = 3, r = 5 (tidak valid)	tidak valid (harus 0 <= r <= n)	Sesuai. r lebih besar dari n.
6	n = -1, r = 0 (tidak valid)	tidak valid (harus 0 <= r <= n)	Sesuai. n negatif.

F. Analisis

Fungsi validNR() dipakai bersama oleh permutasi() dan kombinasi() agar validasi tidak ditulis dua kali. Jika masukan tidak valid, kedua fungsi mengembalikan -1 sebagai penanda, dan main() menampilkan pesan kesalahan.

Fungsi tidak menghitung faktorial penuh, melainkan mengalikan hanya r faktor sehingga risiko overflow lebih kecil. Pada kombinasi(), nilai r diganti n − r bila lebih kecil karena C(n, r) = C(n, n − r), dan pembagian dengan i pada tiap langkah selalu habis karena hasil sementara adalah koefisien binomial bilangan bulat.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala yang diantisipasi adalah overflow bila n besar jika memakai rumus faktorial langsung; ini diatasi dengan perkalian bertahap dan memakai tipe long long. Selain itu, nilai kembalian -1 harus dicek lebih dulu oleh pemanggil sebelum dipakai.

4.10 Tugas 6.5.4 – Konversi suhu

A. Tujuan

Membuat fungsi konversiSuhu(suhu, asal, tujuan) untuk mengonversi suhu antara Celsius, Fahrenheit, dan Réaumur, dengan hasil ditampilkan dua angka desimal.

B. Algoritma atau Langkah Kerja

1. Ubah suhu asal ke Celsius: C = suhu (jika 'C'); C = (suhu − 32) × 5/9 (jika 'F'); C = suhu × 5/4 (jika 'R'). Selain itu kembalikan NAN.
2. Ubah Celsius ke satuan tujuan: C (tetap); F = C × 9/5 + 32; R = C × 4/5. Selain itu kembalikan NAN.
3. Di main(), atur cout dengan fixed dan setprecision(2).
4. Untuk tiap data uji, panggil fungsi; jika hasil NAN cetak "satuan tidak valid", jika tidak cetak hasilnya.

C. Kode program

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>

double konversiSuhu(double suhu, char asal, char tujuan) {
    double c;
    switch (asal) {
        case 'C': c = suhu; break;
        case 'F': c = (suhu - 32.0) * 5.0 / 9.0; break;
        case 'R': c = suhu * 5.0 / 4.0; break;
        default:  return NAN;
    }
    switch (tujuan) {
        case 'C': return c;
        case 'F': return c * 9.0 / 5.0 + 32.0;
        case 'R': return c * 4.0 / 5.0;
        default:  return NAN;
    }
}
int main() {
    struct Uji { double s; char a, t; } uji[] = {
        {100,'C','F'}, {100,'C','R'}, {32,'F','C'}, {80,'R','C'},
        {-40,'C','F'}, {0,'C','C'}, {50,'K','C'}
    };
    std::cout << std::fixed << std::setprecision(2);
    for (auto &u : uji) {
        double h = konversiSuhu(u.s, u.a, u.t);
        std::cout << u.s << " " << u.a << " -> " << u.t << " = ";
        if (std::isnan(h)) std::cout << "satuan tidak valid\n";
        else std::cout << h << '\n';
    }
    return 0;
}
```

D. Hasil Eksekusi

```
100.00 C -> F = 212.00
100.00 C -> R = 80.00
32.00 F -> C = 0.00
80.00 R -> C = 100.00
-40.00 C -> F = -40.00
0.00 C -> C = 0.00
50.00 K -> C = satuan tidak valid
```

E. Pengujian

No.	Data uji	Output aktual	Keterangan
1	100 C → F (normal)	100.00 C -> F = 212.00	Sesuai. 100 × 9/5 + 32 = 212.
2	100 C → R (normal)	100.00 C -> R = 80.00	Sesuai. 100 × 4/5 = 80.
3	32 F → C (titik beku air)	32.00 F -> C = 0.00	Sesuai. (32 − 32) × 5/9 = 0.
4	80 R → C (normal)	80.00 R -> C = 100.00	Sesuai. 80 × 5/4 = 100.
5	-40 C → F (batas, titik temu)	-40.00 C -> F = -40.00	Sesuai. Pada -40, Celsius dan Fahrenheit sama.
6	0 C → C (satuan sama)	0.00 C -> C = 0.00	Sesuai.
7	50 K → C (satuan tidak valid)	50.00 K -> C = satuan tidak valid	Sesuai. Satuan K tidak didukung.

F. Analisis

Fungsi konversiSuhu() menerima tiga parameter: nilai suhu (double) serta satuan asal dan tujuan (char). Semua konversi dilakukan lewat Celsius sebagai perantara, sehingga cukup tiga rumus ke Celsius dan tiga rumus dari Celsius, bukan sembilan rumus terpisah.

Rumus yang dipakai: C = (F − 32) × 5/9; C = R × 5/4; F = C × 9/5 + 32; R = C × 4/5. Satuan yang tidak dikenal menghasilkan NAN, yang dideteksi dengan std::isnan() di main(). Manipulator fixed dan setprecision(2) membuat semua hasil tampil dengan dua angka desimal.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Perlu menyertakan <cmath> untuk NAN dan isnan, serta <iomanip> untuk setprecision. Pada rumus harus ditulis 5.0 / 9.0 (bukan 5 / 9), karena pembagian bilangan bulat menghasilkan 0 dan membuat hasil konversi salah.

4.11 Tugas 6.5.5 – Bilangan prima

A. Tujuan

Membuat fungsi bool prima(int n) yang menentukan apakah suatu bilangan prima, dan mengujinya dengan nilai kurang dari 2, bilangan prima, dan bilangan komposit.

B. Algoritma atau Langkah Kerja

1. Jika n < 2, kembalikan false.
2. Untuk i = 2 selama i × i ≤ n: jika n habis dibagi i, kembalikan false.
3. Jika tidak ada pembagi ditemukan, kembalikan true.
4. Di main(), uji n = -5, 0, 1, 2, 3, 15, 17, 97, dan 100; cetak "prima" atau "bukan prima".

C. Kode program

```cpp
#include <iostream>

bool prima(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) return false;
    return true;
}
int main() {
    int uji[] = {-5, 0, 1, 2, 3, 15, 17, 97, 100};
    for (int n : uji)
        std::cout << n << (prima(n) ? " : prima" : " : bukan prima") << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
-5 : bukan prima
0 : bukan prima
1 : bukan prima
2 : prima
3 : prima
15 : bukan prima
17 : prima
97 : prima
100 : bukan prima
```

E. Pengujian

No.	Data uji	Output aktual	Keterangan

1	-5, 0, 1 (kurang dari 2)	-5 : bukan prima 0 : bukan prima 1 : bukan prima	Sesuai. Bilangan prima dimulai dari 2.
2	2 dan 3 (prima terkecil, batas)	2 : prima 3 : prima	Sesuai. Perulangan tidak berjalan dan fungsi mengembalikan true.
3	17 dan 97 (prima)	17 : prima 97 : prima	Sesuai. Tidak ada pembagi sampai √n.
4	15 dan 100 (komposit)	15 : bukan prima 100 : bukan prima	Sesuai. 15 habis dibagi 3 dan 100 habis dibagi 2.

F. Analisis

Fungsi prima() menerima satu parameter int dan mengembalikan bool. Bilangan kurang dari 2 langsung ditolak karena bukan prima. Untuk bilangan lain, fungsi mencoba pembagi dari 2 dan berhenti begitu ditemukan pembagi, sehingga mengembalikan false lebih awal.

Pembagi hanya perlu dicoba sampai √n, yaitu selama $i \times i \leq n$, karena jika n punya pembagi lebih besar dari √n maka pasangan pembaginya pasti lebih kecil dari √n. Cara ini jauh lebih hemat daripada mencoba semua bilangan sampai $n - 1$.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Perlu memperhatikan nilai 0, 1, dan negatif yang harus ditolak di awal. Selain itu, $i \times i$ dapat melebihi batas int untuk n yang sangat besar mendekati 2.147.483.647; untuk kasus itu dapat dipakai tipe long long.

4.12 Tugas 6.5.6 – Variabel otomatis dan static

A. Tujuan

Membuat dua fungsi yang masing-masing memakai variabel otomatis dan variabel static, memanggil setiap fungsi lima kali, serta menjelaskan perbedaan keluarannya.

B. Algoritma atau Langkah Kerja

1. fungsiOtomatis(): deklarasikan int hitung = 0, naikkan dengan ++, cetak nilainya.
2. fungsiStatic(): deklarasikan static int hitung = 0, naikkan dengan ++, cetak nilainya.
3. Di main(), cetak judul, lalu panggil fungsiOtomatis() lima kali dengan perulangan.
4. Cetak judul kedua, lalu panggil fungsiStatic() lima kali dengan perulangan.
5. Bandingkan keluaran kedua fungsi.

C. Kode program

```cpp
#include <iostream>

void fungsiOtomatis() {
    int hitung = 0;
    ++hitung;
    std::cout << "otomatis: " << hitung << '\n';
}

void fungsiStatic() {
    static int hitung = 0;
    ++hitung;
    std::cout << "static  : " << hitung << '\n';
}
int main() {
    std::cout << "--- fungsiOtomatis ---\n";
    for (int i = 0; i < 5; ++i) fungsiOtomatis();
    std::cout << "--- fungsiStatic ---\n";
    for (int i = 0; i < 5; ++i) fungsiStatic();
    return 0;
}
```

D. Hasil Eksekusi

```
--- fungsiOtomatis ---
otomatis: 1
otomatis: 1
otomatis: 1
otomatis: 1
otomatis: 1
--- fungsiStatic ---
static  : 1
static  : 2
static  : 3
static  : 4
static  : 5
```

E. Pengujian

No	Data uji	Output aktual	keterangan

1	fungsiOtomatis(), pemanggilan ke-1	otomatis: 1	Sesuai.
2	fungsiOtomatis(), pemanggilan ke-5	otomatis: 1	Sesuai. Nilai tidak bertambah antar pemanggilan.
3	fungsiStatic(), pemanggilan ke-1	static : 1	Sesuai. Inisialisasi hanya sekali.
4	fungsiStatic(), pemanggilan ke-3	static : 3	Sesuai.
5	fungsiStatic(), pemanggilan ke-5	static : 5	Sesuai. Nilai bertambah mengikuti jumlah pemanggilan.

F. Analisis

Pada fungsiOtomatis(), variabel hitung dibuat ulang bernilai 0 pada setiap pemanggilan, lalu dinaikkan menjadi 1 dan hilang saat fungsi selesai. Karena itu kelima pemanggilan selalu mencetak 1.

Pada fungsiStatic(), variabel hitung berdurasi sepanjang program dan hanya diinisialisasi satu kali, sehingga nilainya bertahan antar pemanggilan dan bertambah 1, 2, 3, 4, 5. Perbedaan ini menunjukkan bahwa variabel static menyimpan keadaan di dalam fungsi tanpa harus memakai variabel global.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kesalahan konsep yang sering terjadi adalah mengira static int hitung = 0 akan mengatur ulang nilai di setiap pemanggilan; padahal inisialisasi hanya dijalankan satu kali.

4.13 Tugas 6.5.7 – Nilai terbesar

A. Tujuan

Membuat fungsi terbesarTiga(a, b, c) yang mengembalikan bilangan terbesar dari tiga bilangan tanpa menggunakan array.

B. Algoritma atau Langkah Kerja

1. Set maks = a.
2. Jika b > maks, set maks = b.
3. Jika c > maks, set maks = c.
4. Kembalikan maks.
5. Di main(), uji enam kelompok tiga bilangan dan cetak hasilnya.

C. Kode program

```cpp
#include <iostream>

double terbesarTiga(double a, double b, double c) {
    double maks = a;
    if (b > maks) maks = b;
    if (c > maks) maks = c;
    return maks;
}
int main() {
    double uji[][3] = {{3,9,5},{7,7,7},{-1,-5,-3},{1,2,9},{9,2,1},{0,-2.5,-0.5}};
    for (auto &u : uji)
        std::cout << "terbesarTiga(" << u[0] << ", " << u[1] << ", " << u[2]
                  << ") = " << terbesarTiga(u[0], u[1], u[2]) << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
terbesarTiga(3, 9, 5) = 9
terbesarTiga(7, 7, 7) = 7
terbesarTiga(-1, -5, -3) = -1
terbesarTiga(1, 2, 9) = 9
terbesarTiga(9, 2, 1) = 9
terbesarTiga(0, -2.5, -0.5) = 0
```

E. Pengujian

No	**Data uji**	**Output aktual**	**Keterangan**
1	(3, 9, 5) (terbesar di tengah)	terbesarTiga(3, 9, 5) = 9	Sesuai.
2	(7, 7, 7) (semua sama, batas)	terbesarTiga(7, 7, 7) = 7	Sesuai.
3	(-1, -5, -3) (semua negatif)	terbesarTiga(-1, -5, -3) = -1	Sesuai. -1 adalah yang terbesar.
4	(1, 2, 9) (terbesar di akhir)	terbesarTiga(1, 2, 9) = 9	Sesuai.

5	(9, 2, 1) (terbesar di awal)	terbesarTiga(9, 2, 1) = 9	Sesuai.
6	(0, -2.5, -0.5) (desimal dan nol)	terbesarTiga(0, -2.5, -0.5) = 0	Sesuai.

F. Analisis

Fungsi terbesarTiga() menerima tiga parameter double dan mengembalikan satu nilai double. Nilai pertama dijadikan kandidat terbesar, lalu dibandingkan satu per satu dengan b dan c; kandidat diganti hanya bila ada nilai yang lebih besar. Tidak diperlukan array karena jumlah datanya tetap tiga.

Hasil tetap benar untuk berbagai posisi nilai terbesar (awal, tengah, akhir), nilai sama, bilangan negatif, dan desimal. Tipe double dipilih agar fungsi dapat menangani bilangan bulat maupun pecahan.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala logika yang mungkin muncul adalah memberi nilai awal maks = 0, yang salah jika ketiga bilangan negatif. Hal ini dihindari dengan menginisialisasi maks = a.

4.14 Tugas 6.5.8 – Validasi segitiga

A. Tujuan

Membuat fungsi validSegitiga(a, b, c) yang memeriksa sisi positif dan ketaksamaan segitiga, serta menentukan jenis segitiganya.

B. Algoritma atau Langkah Kerja

1. validSegitiga(a, b, c): jika ada sisi ≤ 0 kembalikan false.
2. Kembalikan true hanya jika a + b > c, a + c > b, dan b + c > a.
3. jenisSegitiga(a, b, c): jika tidak valid kembalikan "tidak valid".
4. Jika a = b = c kembalikan "sama sisi"; jika ada dua sisi sama kembalikan "sama kaki"; selain itu kembalikan "sembarang".
5. Di main(), uji tujuh kelompok sisi dan cetak status valid dan jenisnya.

C. Kode program

```cpp
#include <iostream>
#include <string>

bool validSegitiga(double a, double b, double c) {
    if (a <= 0 || b <= 0 || c <= 0) return false;
    return (a + b > c) && (a + c > b) && (b + c > a);
}
std::string jenisSegitiga(double a, double b, double c) {
    if (!validSegitiga(a, b, c)) return "tidak valid";
    if (a == b && b == c) return "sama sisi";
    if (a == b || b == c || a == c) return "sama kaki";
    return "sembarang";
}
int main() {
    double uji[][3] = {{3,4,5},{5,5,5},{5,5,8},{1,2,3},{0,4,5},{-3,4,5},{2,3,10}};
    for (auto &u : uji)
        std::cout << "(" << u[0] << ", " << u[1] << ", " << u[2] << ") -> "
                  << (validSegitiga(u[0],u[1],u[2]) ? "valid, " : "tidak valid, ")
                  << jenisSegitiga(u[0],u[1],u[2]) << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
(3, 4, 5) -> valid, sembarang
(5, 5, 5) -> valid, sama sisi
(5, 5, 8) -> valid, sama kaki
(1, 2, 3) -> tidak valid, tidak valid
(0, 4, 5) -> tidak valid, tidak valid
(-3, 4, 5) -> tidak valid, tidak valid
(2, 3, 10) -> tidak valid, tidak valid
```

E. Pengujian

No	Data uji	Output aktual	Keterangan
1	(3, 4, 5) (normal)	(3, 4, 5) -> valid, sembarang	Sesuai. Semua sisi berbeda.
2	(5, 5, 5) (normal)	(5, 5, 5) -> valid, sama sisi	Sesuai.
3	(5, 5, 8) (normal)	(5, 5, 8) -> valid, sama kaki	Sesuai. 5 + 5 > 8.
4	(1, 2, 3) (batas)	(1, 2, 3) -> tidak valid, tidak valid	Sesuai. 1 + 2 = 3 tidak lebih besar dari 3.
5	(0, 4, 5) (sisi nol)	(0, 4, 5) -> tidak valid, tidak valid	Sesuai. Sisi harus positif.

6	(-3, 4, 5) (sisi negatif)	(-3, 4, 5) -> tidak valid, tidak valid	Sesuai.
7	(2, 3, 10) (ketaksamaan gagal)	(2, 3, 10) -> tidak valid, tidak valid	Sesuai. 2 + 3 < 10.

F. Analisis

Fungsi validSegitiga() mengembalikan bool setelah memeriksa dua syarat: semua sisi positif dan ketiga ketaksamaan segitiga terpenuhi (jumlah dua sisi harus lebih besar dari sisi ketiga). Fungsi jenisSegitiga() memanggil validSegitiga() lebih dulu, sehingga jenis hanya ditentukan untuk segitiga yang sah.

Pada (1, 2, 3), jumlah dua sisi terpendek sama dengan sisi terpanjang sehingga titik-titiknya sejajar dan tidak membentuk segitiga. Pemeriksaan sama sisi dilakukan lebih dulu daripada sama kaki agar segitiga sama sisi tidak salah diklasifikasikan sebagai sama kaki.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Perlu menyertakan <string> karena fungsi jenisSegitiga() mengembalikan std::string. Kendala logika yang diantisipasi adalah memakai >= pada ketaksamaan segitiga, yang akan salah menganggap (1, 2, 3) sebagai segitiga.

4.15 Tugas 6.5.9 – Rekursi

A. Tujuan

Membuat fungsi rekursif jumlahDigit(n) untuk menghitung jumlah digit bilangan bulat nonnegatif, lalu membandingkannya dengan solusi iteratif.

B. Algoritma atau Langkah Kerja

1. Kasus dasar: jika n < 0 kembalikan -1; jika n < 10 kembalikan n.
2. Kasus rekursif: kembalikan (n mod 10) + jumlahDigit(n / 10).
3. Versi iteratif: jumlah = 0; selama n > 0, tambahkan n mod 10 ke jumlah lalu set n = n / 10; kembalikan jumlah.
4. Di main(), uji n = 0, 7, 12345, 9999, 1000, dan -12; cetak hasil rekursif dan iteratif.

C. Kode program

```cpp
#include <iostream>

int jumlahDigit(int n) {
    if (n < 0) return -1;
    if (n < 10) return n;
    return n % 10 + jumlahDigit(n / 10);
}

int jumlahDigitIteratif(int n) {
    if (n < 0) return -1;
    int jumlah = 0;
    while (n > 0) { jumlah += n % 10; n /= 10; }
    return jumlah;
}
int main() {
    int uji[] = {0, 7, 12345, 9999, 1000, -12};
    for (int n : uji)
        std::cout << n << " : rekursif=" << jumlahDigit(n)
                  << ", iteratif=" << jumlahDigitIteratif(n) << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
0 : rekursif=0, iteratif=0
7 : rekursif=7, iteratif=7
12345 : rekursif=15, iteratif=15
9999 : rekursif=36, iteratif=36
1000 : rekursif=1, iteratif=1
-12 : rekursif=-1, iteratif=-1
```

E. Pengujian

No	Data uji	Ouput aktual	keterangan
1	0 (batas)	0 : rekursif=0, iteratif=0	Sesuai.
2	7 (satu digit)	7 : rekursif=7, iteratif=7	Sesuai.
3	12345 (normal)	12345 (normal)	Sesuai. 1 + 2 + 3 + 4 + 5 = 15.
4	9999 (digit berulang)	9999 : rekursif=36, iteratif=36	Sesuai. 4 × 9 = 36.
5	1000 (banyak nol)	1000 : rekursif=1, iteratif=1	Sesuai.
6	-12 (tidak valid)	-12 : rekursif=-1, iteratif=-1	Sesuai. Masukan negatif ditolak.

F. Analisis

Pada versi rekursif, fungsi memanggil dirinya sendiri dengan n / 10 sehingga satu digit terakhir (n % 10) dilepas di setiap pemanggilan. Rekursi berhenti pada kasus dasar n < 10, yaitu ketika tinggal satu digit. Tanpa kasus dasar, fungsi akan memanggil dirinya tanpa henti.

Versi iteratif menggunakan perulangan while dengan kondisi berhenti n > 0 dan menghasilkan nilai yang sama pada semua data uji. Rekursi menambah satu stack frame untuk tiap digit sehingga memakai memori sebanding jumlah digit, sedangkan iterasi hanya memakai variabel tetap; sebaliknya kode rekursif lebih singkat dan mengikuti definisi matematisnya.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Kendala yang diantisipasi adalah rekursi tak berhingga jika kasus dasar tidak ada, serta bilangan negatif yang ditangani dengan mengembalikan -1 sebagai penanda masukan tidak valid.

4.16 Tugas 6.5.10 – Analisis digit

A. Tujuan

Membuat fungsi hitungDigit(n), balikDigit(n), dan palindrom(n), serta mengujinya dengan nol, bilangan satu digit, dan digit berulang.

B. Algoritma atau Langkah Kerja

1. hitungDigit(n): jika n < 0 ubah menjadi -n; banyak = 1; selama n ≥ 10, set n = n / 10 dan tambah banyak; kembalikan banyak.
2. balikDigit(n): balik = 0; selama n > 0, set balik = balik × 10 + n mod 10 dan n = n / 10; kembalikan balik.
3. palindrom(n): kembalikan true jika n ≥ 0 dan n sama dengan balikDigit(n).
4. Di main(), uji n = 0, 7, 121, 1221, 123, 1000, 1111, dan -121; cetak jumlah digit, hasil pembalikan, dan status palindrom.

C. Kode program

```cpp
#include <iostream>

int hitungDigit(int n) {
    if (n < 0) n = -n;
    int banyak = 1;
    while (n >= 10) { n /= 10; ++banyak; }
    return banyak;
}

int balikDigit(int n) {
    int balik = 0;
    while (n > 0) { balik = balik * 10 + n % 10; n /= 10; }
    return balik;
}

bool palindrom(int n) {
    return n >= 0 && n == balikDigit(n);
}
int main() {
    int uji[] = {0, 7, 121, 1221, 123, 1000, 1111, -121};
    for (int n : uji)
        std::cout << n << " : digit=" << hitungDigit(n) << ", balik=" << balikDigit(n)
                  << ", " << (palindrom(n) ? "palindrom" : "bukan palindrom") << '\n';
    return 0;
}
```

D. Hasil Eksekusi

```
0 : digit=1, balik=0, palindrom
7 : digit=1, balik=7, palindrom
121 : digit=3, balik=121, palindrom
1221 : digit=4, balik=1221, palindrom
123 : digit=3, balik=321, bukan palindrom
1000 : digit=4, balik=1, bukan palindrom
1111 : digit=4, balik=1111, palindrom
-121 : digit=3, balik=0, bukan palindrom
```

E. Pengujian

No	Data uji	Output aktual	Keterangan
1	0 (batas, nol)	0 : digit=1, balik=0, palindrom	Sesuai. Nol dihitung 1 digit.
2	7 (satu digit)	7 : digit=1, balik=7, palindrom	Sesuai.
3	121 (palindrom ganjil)	121 : digit=3, balik=121, palindrom	Sesuai.
4	1221 (palindrom genap)	1221 : digit=4, balik=1221, palindrom	Sesuai.
5	123 (bukan palindrom)	123 : digit=3, balik=321, bukan palindrom	Sesuai.
6	1000 (nol di belakang)	1000 : digit=4, balik=1, bukan palindrom	Sesuai. Pembalikan 1000 menjadi 0001 = 1.

7	1111 (digit berulang)	1111 : digit=4, balik=1111, palindrom	Sesuai.
8	-121 (negatif)	-121 : digit=3, balik=0, bukan palindrom	Sesuai. Bilangan negatif tidak dianggap palindrom.

F. Analisis

hitungDigit() membagi n dengan 10 berulang kali dan menghitung berapa kali pembagian dilakukan; nilai awal banyak = 1 membuat angka 0 dan satu digit tetap dihitung 1 digit. balikDigit() mengambil digit terakhir dengan n % 10, menambahkannya ke hasil yang digeser satu tempat (× 10), lalu membuang digit itu dari n.

Fungsi palindrom() memanggil balikDigit() dan memeriksa apakah bilangan sama dengan kebalikannya. Pada 1000 hasil pembalikannya 1 sehingga bukan palindrom, dan bilangan negatif ditolak lebih dulu oleh syarat n >= 0. Hasil seluruh data uji sesuai harapan.

G. Kendala dan Perbaikan

Tidak ditemukan kesalahan kompilasi. Perlu diperhatikan bahwa balikDigit() dapat overflow pada int untuk bilangan 10 digit seperti 1999999999 yang pembalikannya melebihi batas int; untuk kasus itu dapat dipakai long long. Bilangan negatif ditangani dengan mengabaikan tanda pada hitungDigit() dan menolaknya pada palindrom().

BAB V. KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil praktikum, program berjalan dengan baik dan benar. Fungsi memungkinkan program dibagi menjadi bagian kecil yang masing-masing punya tujuan jelas, sehingga kode lebih rapi, tidak berulang, dan mudah diuji. Prototipe memberi tahu compiler antarmuka fungsi sehingga definisinya boleh diletakkan setelah main().

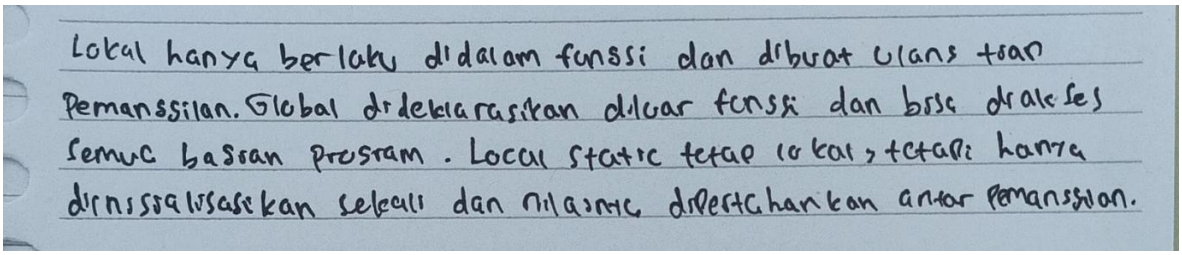
Parameter menerima salinan nilai dari argumen, dan return mengirim hasil kembali ke pemanggil. Variabel lokal otomatis dibuat ulang setiap pemanggilan, sedangkan variabel lokal static mempertahankan nilainya antar pemanggilan.

5.2 Saran

Tidak ada, sudah bagus pak 👍

JAWABAN 10 SOAL ANALISIS

1. Apa perbedaan prototipe fungsi dan definisi fungsi?
   Jawaban : Prototipe hanya deklarasi (tipe balik, nama, parameter, diakhiri `;`) supaya fungsi bisa dipanggil sebelum didefinisikan. Definisi berisi badan fungsi yang benar-benar dijalankan.
2. Apa perbedaan parameter dan argumen?
   Jawaban : Parameter adalah variabel di definisi fungsi. Argumen adalah nilai yang dikirim saat fungsi dipanggil. Pada `jumlahSampai(10)`, 10 adalah argumen dan `n` adalah parameter.
3. Mengapa fungsi yang mengembalikan nilai menggunakan return?
   Jawaban : `return` mengirim hasil ke pemanggil dan menghentikan fungsi, sehingga hasilnya bisa disimpan atau dipakai dalam ekspresi.
4. Apa risiko jika fungsi bertipe int tidak mengembalikan nilai pada jalur eksekusi yang seharusnya?
   Jawaban : Perilakunya tidak terdefinisi (*undefined behavior*): nilai yang diterima bisa acak sehingga hasil salah. Compiler biasanya memberi peringatan.
5. Jelaskan perbedaan variabel lokal, global, dan lokal static.
   Jawaban : Lokal hanya berlaku di dalam fungsi dan dibuat ulang tiap pemanggilan. Global dideklarasikan di luar fungsi dan bisa diakses semua bagian program. Lokal static tetap lokal, tetapi hanya diinisialisasi sekali dan nilainya dipertahankan antar pemanggilan.



6. Mengapa penggunaan variabel global secara berlebihan menyulitkan pemeliharaan program?
   Jawaban : Nilainya bisa diubah dari mana saja, jadi sulit dilacak dan diuji, serta rawan efek samping dan konflik nama.
7. Apa keuntungan membagi program menjadi beberapa fungsi dibanding menulis seluruh instruksi di main()?
   Jawaban : Kode lebih rapi dan mudah dibaca, tidak berulang (bisa dipakai ulang), serta lebih mudah diuji dan diperbaiki.
8. Mengapa fungsi akar kuadrat perlu menangani masukan negatif?
   Jawaban : Akar bilangan negatif tidak ada dalam bilangan real. Pada metode Newton, perulangan tidak akan pernah berhenti untuk x negatif.
9. Bandingkan pendekatan iteratif dan rekursif dari kondisi berhenti serta penggunaan sumber daya.
   Jawaban : Iteratif berhenti saat kondisi perulangan salah dan memakai memori tetap. Rekursif berhenti di kasus dasar dan memakai memori tambahan (*stack*) tiap pemanggilan, sehingga bisa *stack overflow* kalau terlalu dalam.

10. Pilih satu tugas, lalu jelaskan tiga kasus uji: normal, batas, dan tidak valid, beserta keluaran yang diharapkan.
    Jawaban :

Tugas 6.5.5 (prima):

- Normal: `17` → prima.
- Batas: `2` → prima.
- Tidak valid: `-5` → bukan prima (karena < 2)

DAFTAR PUSTAKA

Cantumkan sumber teori yang benar-benar digunakan.

Modul praktikum pemrograman 6 : MENGEMBANGKAN FUNGSI-FUNGSI SENDIRI

LAMPIRAN

- Lampiran A: Kode sumber program `.cpp`.
- Lampiran B: Tangkapan layar kompilasi dan hasil eksekusi.
- Lampiran C: Berkas tambahan apabila diminta dosen.